SVERRE HENDSETH

MARBLE DOCUMENTATION: A SOUND TUTORIAL

Contents

Introduction 4

Hello World 4

Generating sound: The `TuningFork` *model* 5

- *Defining a* `Waveform` *and* `TuningFork` *objects* 5
- *Completing the model* 6

A hierarchical model: Melody, Lines, Bars, Notes 7

- *Making a simple model of a Melody* 7
- *Using the Melody hierarchy: Adding a tuning fork to to each note* 8
- *Using the Melody hierarchy: Printing the score* 10
- *Drawing the object graph* 13

Modelling an instrument 14

- *The Instrument* 14
- *Testing instruments* 16
- *Letting the instrument play the melody* 16

Making variants of an instrument 17

Improving the melody with self-similarity 19

The Grail: Combining trees, and a marble module in Marble 20

- *Introduction* 20
- *Step 1: Generating some lyrics* 21
- *Step 2: The specification: How the phase 2 model can be used* 23
- *Step 3: The phase 2 module* 24
- *The result: Running the specification test* 26
- *Step 4: Generating marble models from marble - The* $marblemod_{marble.marble}$ *module.* 26

Summary: Selling points 29

Appendix: The dot/graphwiz model 29

Appendix: The Makefile 30

Introduction

You will be needing `marbleapp_playsound` and `marbleapp_echo` for this tutorial

This tutorial is not about making music. Its primary purpose is to give a picture of the expressive power of the marble modelling language.

Explain clip features.

Status: Leaks memory

- Uppercase letter starts class.

Hello World

This is a complete Marble model; it is made to emit "Hello World":

```
/*** #file "hello.marble" ***/
Output {}

System {
  Collection(collectionName,*|Output|line);
  o = Output {
    line = "Hello World\n";
  }
}
/*** End of File ***/
```

We notice the following:

- We define two classes `Output` and `System`.
- `Output` has no properties given by its original definision, but is given a `line` text variable when an instance of it is created as a subobject of `System`.
- `System` has, in addition to the `Output` subobject, also a named 'collection'. The collection provides a *pattern* (`'*|Output|line'`) that, when matched, will initiate a callback to the host program. The host program will initiate a traversal of the object hierarchy by specifying the root object class (`System`) and the collection name (`collectionName`). While traversing the object hierarchy, every time we encounter an object of type `Object` with a property called `line`, a callback will be made with the value of the `line` variable.

`marbleapp_echo` is one such host program. Call it like this:

```
marbleapp_echo -f hello.marble
```

It will read the given model, and ask for the `System` object to be traversed, asking for `collection` callbacks to be made.

The callback function in `marbleapp_echo` will just echo the "Hello World" string to the console/stdout.

Generating sound: The `TuningFork` model

Defining a `Waveform` and `TuningFork` objects

Marble lets us define classes with properties.

For example:

```
Waveform {
  amplitude = 100;
  value = amplitude * sin(freq * ^time * 2 * m_PI);
}
```

is a valid object description giving defining a object called `Waveform` with properties called `amplitude` and `value`. Some points need explaining:

Where do `m_PI` come from? `m_PI` is in fact hardcoded into the language along with the `sin()` function... Be aware that as per now the language is in an experimental stage, and the APIs are not well harmonized.

Where do `freq` come from? We do not know yet; this is not a complete model. There are three possibilities:

- The least interesting is that it can be defined in another definision of `Waveform`:

  ```
  Waveform {
    freq = 440;
  }
  ```

  Great; these two definisions of `Waveform` are just merged, with the later overriding the earlier if there are redefinitions or conflicts.

- A more interesting alternative comes from the fact that objects can have other objects as properties: Let us say that a `TuningFork` have a `Waveform` object to generate its waveform.

  ```
  TuningFork {
    waveform = Waveform {
      freq = 440;
    }
  }
  ```

We see that the `Waveform` object created for the `TuningFork` gets it frequency decided when the subobject is defined.

- The third alternative for defining the frequency is to set the `freq` of *any* `waveform` objects that Tuningfork owns.

```
TuningFork {
  waveform|freq = 440;
}
```

Here any object created for the `waveform` property gets the freq property defined. Wildcards like '*' can also be used.

Where does `time` come from and what does the ^ symbol mean? '^' is the dynamic scope operator and means that the accessed property is not found in the `Sinus` object itself, but in an owning object. That is, first `TuningFork` is examined to see if it has a time property, and if not *its* owning object is examined and so on. We say that time is 'pulled' into the `Waveform` object.

As it is now, the tuning fork just models a steady 440Hz sinus wave. Let us also model how the strength of the sound varies over time using a standard Attack/Decay/Sustain/Release (ADSR) envelope. Here we must manipulate the `amplitude` of the `Waveform`:

```
TuningFork {
  envelope = [[0,0],[0.05,120],[0.1,100],[0.5,100],[3,0]];
  waveform = Waveform {
    amplitude = Interpolate(<envelope,^time);
  }
}
```

The tuning fork will sound for 3 seconds, all together.

We see the definision of arrays, even two-dimensional, and the use of the `Interpolate` predefined function.

We also see a new scope operator '<'. This is the lexical scope — the context that you can see in the text editor.

We have now two conflicting definisions of `amplitude` for the `Waveform` object: The resolving principle here is that the owning object. *The outermost definision takes priority*. (If there is still a conflict, we already know that a later definition will override the earlier.)

Completing the model

Here is a complete model, split into two files:

```
/*** #file "tuningfork.marble" ***/
Waveform {
  amplitude = 100;
```

```
  value = amplitude * sin(freq * ^time * 2 * m_PI);
}

TuningFork {
  envelope = [[0,0],[0.05,120],[0.1,100],[0.5,100],[3,0]];
  waveform = Waveform {
    freq = 440;
    amplitude = Interpolate(<envelope,^time);
  }
}
/*** End of File ***/
```

```
/*** #file "playtuningfork.marble" ***/
Include tuningfork.marble

System {
  time = 0;
  Collection(value,*|Waveform|value);
  tf = TuningFork {}
}
/*** End of File ***/
```

We see that here the collection is called 'value', and the type of the `value` property will be a number.

The `marbleapp_sound` is a host program that is a bit more complex than `marbleapp_echo`. It loops around the following

- It *sets* the `time` variable in the `System` object.
- It initiates traversing the `System` object, asking for the `value` collection.
- The value given by all the callbacks are added together.

This sequence of sums are then normalized and written to a wav sound file.

Listen (if this local file link works)

A hierarchical model: Melody, Lines, Bars, Notes

Making a simple model of a Melody

Being able to define hierarchies of objects should be well suited to modelling these terms; A melody consists of a number of lines, A line consist of a number of bars and a bar consist of a number of notes. Let us aim for a simple model to start with:

A note has a duration and a pitch.

- We measure the duration in the number of 8th's.
- Let the pitch be described by just an integer.

A bar has a given duration - It should contain notes amounting exactly to this duration.

We could imagine some scheme for varying rythm here, but for now let us keep it simple with four notes with duration 4 and a random pitch.

A line have a number of bars and a melody has a number of lines.

```
/*** #file "melody.marble" ***/
Note {
  pitch = iRand(8);
  duration = 2;
}

Bar {
  notes = for(4,noteN) Note {}
}

Line {
  bars = for(nOfBars,barN) Bar {}
}

Melody {
  barsPerLine = [2,2];
  lines = foreach(barsPerLine,nOfBars,lineN) Line {}
}
/*** End of File ***/
```

Great, we have a model of a melody...

Some few new features here:

- iRand(8) will generate an random integer between 0 and 7.
- We see 'for loops' creating more subobjects in one statement. In fact all subobject variables may well have multiple objects.
- We put this simple model into a file of its own: It may be included in other models.

Using the Melody hierarchy: Adding a tuning fork to to each note

Now, how do we play the melody?

- we could add a `TuningFork` object to each note[1].

[1] Using the old tuningfork model here is probably not the good solution. It is used here, just to demonstrate that it can be done. Later in this tutorial we will let the melody be played by an instrument which is a better way of doing it.

- And decide the freq of the `TuningFork` by looking it up in a Scale table with our pitch integer.
- Also we need some concept of timing': Let us add the starting time and duration as explicit parameters of the tuningfork.

  Let the tempo be 240, four notes per second.

```
/*** #file "playmelody.marble" ***/
Include melody.marble
Include tuningfork.marble

// We need to extend the tuningfork model slightly
TuningFork {
  startTime = 0;
  duration = 0.25;

  Guard(started,^time>startTime);
  Guard(notEnded,^time<startTime+duration);

  // Make the envelope duration dependent
  envelope = [[0,0],[0.05,120],[0.1,100],[duration-0.1,100],[duration,0]];
  // Make the amplitude startTime dependent
  waveform | amplitude = Interpolate(<envelope,^time - <startTime);
}

System {
  time = 0;
  Collection(value,*|Waveform|value);

  scale = [[0,1],[1,pow(2,2/12)],[2,pow(2,4/12)],[3,pow(2,5/12)],
           [4,pow(2,7/12)],[5,pow(2,9/12)],[6,pow(2,11/12)],[7,2]];

  melody = Melody(123,456) {}

  // Add a tuningfork to each Note
  * | Note | t = TuningFork {}
  // Set the time when to start playing
  * | TuningFork | startTime = (^lineN * ^nOfBars * 4 + ^barN * 4 + ^noteN)*0.25;
  // Then set the frequency of the played wave.
  * | Waveform | freq = 880 * Interpolate(^scale,^pitch);
}
/*** End of File ***/
```

  Listen

Most of this should be straightforward now, even though more complex forms of mechanisms already introduced are used.

- We introduce named 'Guards'. If a guard expression is evaluated to false, the object "does not exist": It will not result in callbacks, and its subobjects are not created.

Using the Melody hierarchy: Printing the score

Let us also demonstrate that the same model can be used for generating a lilypond sourcefile that can be used to generate the score.

We will now be using marbleapp$_{\text{echo}}$ again to print the source file.

A sample lilypond file can be a bit of a hassle...

The lilypond module

```
/*** #file "marblemod_lilypond.marble" ***/

Output{}
Empty{}

LilypondPlainMelody {

  scale = ["c'","d'","e'","f'","g'","a'","b'","c''"];
  title = "A Melody";
//  paperSize = "a4";
//  paperOrientation = "portrait";
  composer = "A Marble Model";

  textLines1 += Output {line = "\version \"2.14.2\"\n";}

  // http://lilypond.org/doc/v2.18/Documentation/usage/lilypond-output-in-other-programs
  textLines1 += Output {line = "\paper{";}
  textLines1 += Output {line = "  indent=0\mm";}
  textLines1 += Output {line = "  line-width=120\mm";}
  textLines1 += Output {line = "  oddFooterMarkup=##f";}
  textLines1 += Output {line = "  oddHeaderMarkup=##f";}
  textLines1 += Output {line = "  bookTitleMarkup = ##f";}
  textLines1 += Output {line = "  scoreTitleMarkup = ##f";}
  textLines1 += Output {line = "}";}

//  textLines1 += Output {line = "\paper {\n";}
//  textLines1 += Output {line = "  #(set-paper-size \"" + <paperSize + "\" '" + <paperOrientation + ")\n"
//  textLines1 += Output {line = "}\n";}
```

```
  textLines1 += Output {line = "\header {\n";}
  textLines1 += Output {line = "  title = \"" + <title + "\"\n";}
  textLines1 += Output {line = "  composer = \"" + <composer + "\"\n";}
  textLines1 += Output {line = "}\n";}

  textLines1 += Output {line = "melody = {\n";}
  textLines1 += Output {line = "  \clef treble\n";}
  textLines1 += Output {line = "  \key c \major\n";}
  textLines1 += Output {line = "  \\time 4/4\n";}
  textLines1 += Output {line = "  \\tempo 4 = 240\n";}
  textLines1 += Output {line = "\n";}

  contents = Empty {}

  textLines2 += Output {line = "}\n\n";}

  textLines2 += Output {line = "mylyrics = \lyricmode {\n";}
  lyricscontents = Empty {}
//  textLines2 += Output {line = "  Joy to the world, the Lord is come.\n";}
  textLines3 += Output {line = "}\n";}

  textLines3 += Output {line = "\n";}

  textLines3 += Output {line = "\score {\n";}
  textLines3 += Output {line = "  \\new Staff \melody\n";}
  textLines3 += Output {line = "  \addlyrics \mylyrics\n";}
  textLines3 += Output {line = "  \layout { }\n";}
  textLines3 += Output {line = "  \midi { }\n";}
  textLines3 += Output {line = "}  \n\n";}
}

Note {
  noteName = ^scale[pitch];

  noteString = ["Never Used - A note can not have 0 duration!",
                noteName + "8",
                noteName + "4",
                noteName + "4.",
                noteName + "2",
                noteName + "8~ " + noteName + "2",
                noteName + "2.",
                noteName + "8~ " + noteName + "2.",
                noteName + "1"];
```

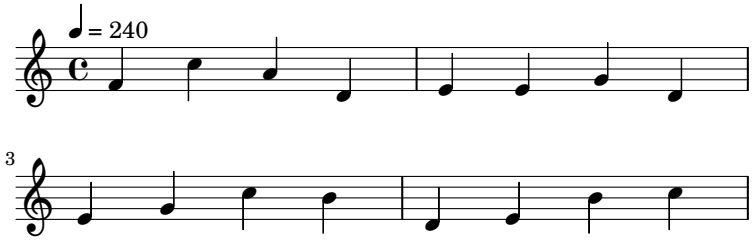
```
  o = Output{line = ^noteString[^duration] + " ";}
}

Bar {
  o = Output{line = " | \n";}
}

Line {
  o = Output{line = " \\break \n";}
}

/*** End of File ***/
```

Some notes:

- Notice the `contents` and `textLines` 1 and 2 trick to ensure that the lines of the lilypond file comes out in the proper order.

printing the score

```
/*** #file "melodyscore.marble" ***/
Include melody.marble
Include marblemod_lilypond.marble

System {
  Collection(collection,*|Output|line);

  melody = LilypondPlainMelody {
    paperSize = "a5";
//    paperOrientation = "landscape";
    contents = Melody(123,456) {}
  }
}

/*** End of File ***/
```



But wait… the printed melody is not the same as the one in the audio file! We are not going deeply into this now, but here are some forward pointers:

- There are two random seeds connected to each object (set by hashing a combination of the owning objects corresponding seed, the index in the object collection, and the class name).
- Try initiating the Melody objects with specific seeds like this:

```
//In playmelody.marble
melody = Melody(123,456) {}
//In melodyscore.marble
contents = Melody(123,456) {}
```

  and you will see that the same melody is generated.

Drawing the object graph

We might also want to plot the object hierarchy with dot/graphviz tools. A module from another project defined the DotModel, DotNode and DotArrow classes. What we need to do is to create a DotModel object and give it the melody. Then all melody objects must named, and DotArrows must be made for all parent/child relationships.

```
/*** #file "melody_dotmodel.marble" ***/
Output{}
Empty{}
Include melody.marble
Include marblemod_graphviz.marble

Melody {
  name = "Melody";
}

Line {
  name = "Line" + "_" + numberFormat("%.0f",lineN);
  parentName = ^name;
  da = DotArrow {
    to = <name;
    from = <parentName;
  }
}

Bar {
  name = "Bar" + "_" + numberFormat("%.0f",^lineN) + "_" + numberFormat("%.0f",barN);
  parentName = ^name;
  da = DotArrow {
    to = <name;
```

```
    from = <parentName;
  }
}

Note {
  name = "Note" + "_" + numberFormat("%.0f",^lineN) + "_" + numberFormat("%.0f",^barN) + "_" + numberFormat
  parentName = ^name;
  da = DotArrow {
    to = <name;
    from = <parentName;
  }
}

System {
  Collection(collection,*|Output|line);

  model = "melody_dotmodel";

  dotModel = DotModel {
    title = "The Melody Object Hierarchy";
  }
  melody = Melody {}
}
/*** End of File ***/
```

...and here it is:

The Melody Object Hierarchy

Modelling an instrument

We will continue with the melody object from the previous chapter.

Playing the melody with pure Sine wawes is a bit boring. Let us make a bit more complex model of an Instrument.

Rather than "giving a tuning fork to all notes" we will now "give notes to the instrument".

The Instrument

To make instruments different let us add a numer of overtones to the base frequency, some vibrato, and add a multiplier to the octave, and some slight errors in intonation.

The Notes must be assumed to have a `startTime` and `duration`.

```
/*** #file "instrument.marble" ***/
Waveform {
  amplitude = 100;
  value = amplitude * sin(freq * time * 2 * m_PI);
}

Instrument {
  envelope = [[0,0],[0.05,120],[0.1,100],[0.9,100],[1,0]];

  // Offsets the overtones - with different offsets
  disharmonyLimit = 0.1*fRand();

  // Offsets all - with the same offset
  maxIntonationError = 0.1;
  intonationError = 1+(maxIntonationError*fRand() - maxIntonationError/2);

  octaves = [0.25,0.5,1,2,4,8];
  octaveFactor = octaves[iRand(4)];

  vibrAmp = 0.1*fRand();  // frequency deviance
  vibrFreq = 15*fRand();  // Times a second.

  // An overtone has an amplitudeFactor and a frequencyFactor
  waveformsData = [
    [1/1*fRand(),octaveFactor*1*intonationError],
    [1/2*fRand(),octaveFactor*2*(1+disharmonyLimit*fRand() - disharmonyLimit/2)*intonationError],
    [1/3*fRand(),octaveFactor*3*(1+disharmonyLimit*fRand() - disharmonyLimit/2)*intonationError],
    [1/4*fRand(),octaveFactor*4*(1+disharmonyLimit*fRand() - disharmonyLimit/2)*intonationError],
    [1/5*fRand(),octaveFactor*5*(1+disharmonyLimit*fRand() - disharmonyLimit/2)*intonationError],
    [1/6*fRand(),octaveFactor*6*(1+disharmonyLimit*fRand() - disharmonyLimit/2)*intonationError]
  ];

  * | Note | waveforms = foreach(<waveformsData,waveformData) Waveform {

    amp = waveformData[0];
    amplitude = amp * Interpolate(<envelope,(^time-^startTime)/^duration);

    // Make the vibrato by skewing the time
    time = ^time + ^vibrAmp/baseFreq *sin(^vibrFreq * ^time * 2 * m_PI);


    baseFreq = 440 * ^scale[^pitch] * waveformData[1];
```

```
    freq = baseFreq;
  }
}
/*** End of File ***/
```

Testing instruments

As a test we make a number of different Instruments and give them a note each to play.

```
/*** #file "instrumenttest.marble" ***/
Output{}
Empty{}
Include melody.marble // To get the note
Include instrument.marble

System {
  time = 0;
  Collection(value,*|Waveform|value);

  scale = [1,pow(2,2/12),pow(2,4/12),pow(2,5/12),
           pow(2,7/12),pow(2,9/12),pow(2,11/12),2];

  instruments = for(12,instrumentN) Instrument {
    n = Note {
      startTime = <instrumentN*0.25;
      duration = 0.25;
      pitch = 0;
      Guard(started,^time > startTime);
      Guard(notEnded,^time < startTime+duration);
    }
  }
}
/*** End of File ***/
```

Letting the instrument play the melody

Great. And now we can give a melody to an object for playing:

```
/*** #file "instrumentmelody.marble" ***/
Output{}
Empty{}
Include melody.marble
Include instrument.marble
```

```
Note {
  startTime = (^lineN * ^nOfBars * 4 + ^barN * 4 + noteN)*0.25;
  duration = 0.25;
  Guard(started,^time > startTime);
  Guard(notEnded,^time < startTime+duration);
}

System {
  time = 0;
  Collection(value,*|Waveform|value);

  scale = [1,pow(2,2/12),pow(2,4/12),pow(2,5/12),
           pow(2,7/12),pow(2,9/12),pow(2,11/12),2];

  instruments = for(2,i) Instrument {
    m = Melody{}
  }
}
/*** End of File ***/
```

Making variants of an instrument

But let us say that we wanted this two-part melody played not with two completely different instruments, but with two instruments of the "same type". The octave and amplitude of the overtones should be kept the same, but the rest of the parameters should still be allowed to vary.

Marble allows for this by associating *two* seeds to each object. Wich seed is used is depending on which kind of assignment statements we are using. Enter the "=!" assignment operator:

```
var1 =! iRand(1000);
var2 =  iRand(1000);
```

These two assignment statements will use the two different seeds.

When defining an object we may give zero, one or both of these seeds.

Note that if the '=!' operator is used once for a variable, that variable is tagged with this seed, and all other statements assigning to or modifying this variable will also use this seed.

```
i1 = Instrument {}
i2 = Instrument(123) {}
i3 = Instrument(123,456) {}
i4 = Instrument {}
i5 = Instrument(123) {}
i6 = Instrument(123,456) {}
```

Here i1 and i4 are completely independent of each other and of the others. They are *Created*.

i2 and i5 will be mainly the same instrument in the sense that every variable assigned with `! will be the same.\footnote{Except if you have 'added in' any random variables defined by ''}` They are *Clones*.

Here i3 and i6 will be mainly the same as i2 and i5, but will be exactly similar to each other. They are *Copies*.

So let us modify the waveformsData and octaveFactor of the instrument.

To Sverre: How does this work when assigning to objectcollections? Must make a test :-)

```
/*** #file "sameinstrumentmelody.marble" ***/
Output{}
Empty{}
Include melody.marble
Include instrument.marble

Note {
  startTime = (^lineN * ^nOfBars * 4 + ^barN * 4 + noteN)*0.25;
  duration = 0.25;
  Guard(started,^time > startTime);
  Guard(notEnded,^time < startTime+duration);
}

Instrument {
  octaveFactor =! octaveFactor;
  waveformsData =! waveformsData;
}

System {
  time = 0;
  Collection(value,*|Waveform|value);

  scale = [1,pow(2,2/12),pow(2,4/12),pow(2,5/12),
           pow(2,7/12),pow(2,9/12),pow(2,11/12),2];

  instruments = for(2,i) Instrument(123) {
    m = Melody{}
  }
}
/*** End of File ***/
```

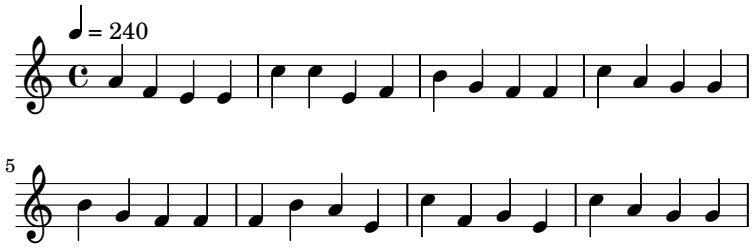
Improving the melody with self-similarity

Let us use this seed manipulation also to improve slightly on the melody. Self-similarity - that there are recognizable patterns in music is good, and here half of the bars will be build on the same relative pitch sequence. [2]

[2] The extra indirection of the TemplateBar seems unnecessary; The language screams for an addition of some elegant if statement.

```
/*** #file "selfsimilar_melodyscore.marble" ***/
Include melody.marble
Include marblemod_lilypond.marble

TemplateBar {
  b = Bar(124) {
    Note | basepitch =! iRand(4);
    pitchOffset = iRand(5);
    Note | pitch = basepitch + <pitchOffset;
  }
}

PossiblyTemplateBar {
  isTemplate = fRand() < 0.5;
  b1 = Bar {
    Guard(deleteIt,1-^isTemplate);
  }
  b2 = TemplateBar {
    Guard(deleteIt,^isTemplate);
  }
}

Line {
  bars = for(nOfBars,barN) PossiblyTemplateBar {}
}

System {
  Collection(collection,*|Output|line);

  * | Melody | nOfLines = 4;
  * | Line | nOfBars = 4;

  melody = LilypondPlainMelody {
    title = "A Melody with some similar bars";
    paperSize = "a6";
    paperOrientation = "landscape";
    contents = Melody {}
  }
```

```
}

/*** End of File ***/
```



Play the midi: `./marbledoc_soundtutorial/selfsimilar_melodyscore.midi`

The Grail: Combining trees, and a marble module in Marble

Introduction

We can easily envision how a `Melody Line` can be improved by controlling the underlying chord sequences and differing between start, middle and end bars, without breaking the basic infrastructure we have now or needing more expressive power in the language. The same goes for building, under each `Bar`, a rythm hierarchy where notes are being subdivided to make more insteresting rythms. Also here we could apply self similarity principles as we did with the pitches. Etc.

But what if we also were generating the lyrics of the melody, where the syllables of the text should correspond to the notes? This would fit into our framework only if breaking down the text into Lines, Bars and Notes/Syllables made sense.

But even though we can reasonably match Lines with sentences and Notes to Syllables, we can not expect this match to be perfect. Melody lyrics would probably require thinking about progression of sentences, semantic meaning, grammar etc. And even if we *could* make the match, we would mix two relatively independent hierarchies into one model, which would be bad form seen from a software/maintenance perspective.

Still the two hierarchies are coupled.

Marble is not aiming to express and resolve two-way dependencies between such trees — The "order of decitions" should be clear and explicit in the complete model. That is; mimicing what happens in the mind of the human songwriter that bulds the lyrics and the melody "at the same time" without being able to put exact words to his strategies, is beyond scope here. Also any trial-and-failure, iterative, equation-solving or optimization processes are not in scope.

We will assume that the hierarchies will interact/influence each other in a limited number of well-defined phases corresponding to the decition order. And to make it simple; Let us generate the lyrics first, and use this to dominate the rythm of the melody.

The strategy will be to have one model generating the lyrics, and then generate a model of the melody that conforms to this specification.[3]

We need:

- A model of how to generate lyrics.
- A module letting us generate Marble models.

These follows the same pattern as when we used the Melody object to generate both the sound, score and hierarchy plot.

- And we need to extend the Melody model to generate rythms according to syllable specification.

[3] I spent a lot of time (years!) on trying to contruct in-language expressive power to handle such phases between interacting models, before I discovered that generating new generations of models was a low-hanging solution.

Step 1: Generating some lyrics

Let us assume here, for simplicity, that lyrics consists of a number of words in random order. The words consists of syllables with different wheights.

- A verse consists of a number of lines with a given number of syllables.
- The line should end with a syllable with heavy wheight.
- Bars should end on word endings.

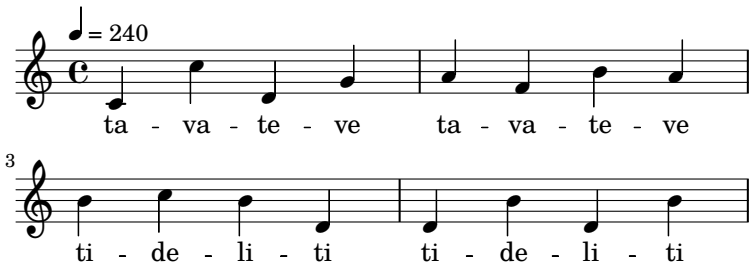
```
/*** #file "lyrics.marble" ***/
Word{
  theseWords = isEndWord? ^endWords[thisWordLength-1] : ^words[thisWordLength-1];
  wordIndex = iRand(arraySize(theseWords));
  thisWord = theseWords[wordIndex];

  syllables = for(arraySize(thisWord[0]),i) Syllable {
    str = <thisWord[0][i];
    weight = <thisWord[1][i];
  }
}
Syllable{ // Why is there necessary here???
  delim = " -- ";
  midText = <<delim?" ";
}
```

```
Output{}

LyricLine {

  words = while(i) Word {
    Guard(stop,pLength > 0);

    pLength = <<newLength ? ^lineLength;
    newLength = pLength - thisWordLength;

    thisWordLength = pLength>4? iRand(4)+1 : pLength;

    isEndWord = newLength>0?0:1;
  }
}

Lyrics {
  lineLengths = [13,9,9,13];

  words = [ [ [["rai"],[4]], [["dom"],[4]], [["ta"],[4]] ],
            [ [["sut","tam"],[2,2]], [["shak","ka"],[2,2]], [["ta","ve"],[2,2]] ],
            [ [["ti","de","li"],[1,1,1]], [["di","de","li"],[1,1,1]] ],
            [ [["ta","va","te","ve"],[1,1,1,1]], [["ti","de","li","ti"],[1,1,1,4]] ]
          ];

  endWords = [ [ [["rai"],[4]], [["dom"],[4]], [["tam"],[4]], [["bom"],[4]] ],
            [ [["ta","daa"],[2,4]] ],
            [ [["dam","di","dam"],[2,1,4]] ],
            [ [["ti","de","li","ti"],[1,1,1,4]] ]
          ];

  lines = foreach(lineLengths,lineLength) LyricLine {}
}
/*** End of File ***/
```

To test it, let us just add lyrics to the old melody score. The Lyrics model creates these syllable objects, but are not yeat equipped to output them in any way, so this must be added.

```
/*** #file "lyricstest.marble" ***/
Include melody.marble
Include marblemod_lilypond.marble
Include lyrics.marble

Syllable {
```

```
  delim = " -- ";
  midText = <<delim?" ";
  // emit the syllable
  o = Output{line = ^midText + ^str;}
}

System {
  Collection(collection,*|Output|line);

  // Add an ending newline only after each LyricLine
  // owned by the lyrics themselves
  Lyrics | LyricLine | ending = Output{line = "\n";}

  melody = LilypondPlainMelody {
    title = "A Melody with lyrics";
    paperSize = "a6";
    paperOrientation = "landscape";
    contents = Melody {
      nOfLines = 4;
      nOfBarsPerLine = 4;
    }
    lyricscontents = Lyrics {
      lineLengths = [16,16,16,16];
    }
  }
}
/*** End of File ***/
```



Step 2: The specification: How the phase 2 model can be used

In the example over the melody and lyrics model are completely independent. Our ambition is, however that the melody should be generated to fit the lyrics, taking into consideration syllable weights, word ends and lines:

- The lines of the lyrics corresponds to the lines of the melody
- The number of bars is decided by some weighed sum of syllables

- Syllables are distributed to bars eagerly, possibly leaving the last bar to be a bit underfilled.
- The rythm inside of a bar can also be made eagerly, by looking at syllable wheight.

So let us imagine this melody model generated by the lyrics model:

```
/*** #file "lyricsmelody_phase2_test.marble" ***/
Include lyricsmelody_phase2.marble

System {
  Collection(collection,*|Output|line);

  wordWeights = [[2,2],[1,1,1,1],[1,1,1,4],[4]];

  lyricsText = "";
  lyricsText = lyricsText + "ta -- ve ta -- va -- te -- ve\n";
  lyricsText = lyricsText + "ti -- de -- li -- ti dom \n";

  m = LyricsMelody_Phase2 {
  }
}
/*** End of File ***/
```

Step 3: The phase 2 module

From this specification we can make the $\text{LyricsMelody}_{\text{Phase2}}$ class, that will generate the melody.

- We can skip the Line abstraction of the Melody, since this is taken care of already by the Lyrics in phase 1.

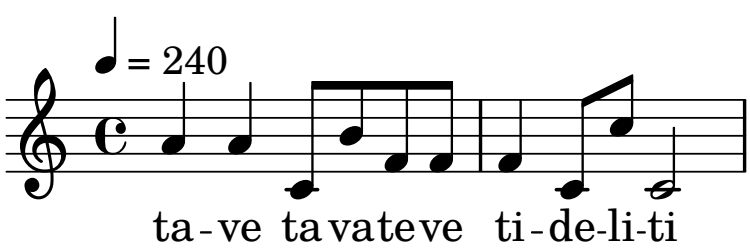
```
/*** #file "lyricsmelody_phase2.marble" ***/
Include melody.marble
Include marblemod_lilypond.marble
Include lyrics.marble

Empty {}
ABar {}

Melody {
  lines = Empty {}          // Drop the old Lines,

  bars = while(nBar) ABar { // ...go directly for Bars
```

```
    Guard(stop,nBar < arraySize(^wordWeights));

    minWeight = 5;

    pWeightsSoFar = <<newWeightsSoFar ? [];
    weightsSoFar = appendArray(pWeightsSoFar,^wordWeights[nBar]);
    newWeightsSoFar = weightSum > minWeight ? [] : weightsSoFar;

    weightSum = arraySum(weightsSoFar);

    bar = Bar {
       Guard(createBar,<weightSum > <minWeight);
       weights = <weightsSoFar;
//       weightSum = <weightSum;
    }
  }
}

Bar {
  restBarLength = 8;
  notesTaken = 0;

  lengthInEights = [0,8,4];

  notes = for(arraySize(weights),i) Note {
    restBarLength = <<newRestBarLength ? 8;
    newRestBarLength = restBarLength - duration;

    restNoteWeight = <<newRestNoteWeight ? ^weightSum;
    newRestNoteWeight = restNoteWeight - ^weights[i];

    // The duration of a note is modified depending on how full the bar is.
    modifiedWeight = arraySize(^weights) == i+1 ?
                 restBarLength : // The last note must take the rest of the bar
                 (restBarLength == restNoteWeight ?
                   ^weights[i] : // If there is a perfect match
                   (restBarLength  > restNoteWeight ?
                     ^weights[i] + 1 : // If we have to little weight, increase
                     ^weights[i] - 1)); // If we have to much weight, decrease
    noteLengths = [1,1,2,3,4,5];
    duration = noteLengths[modifiedWeight];
  }
}
```

```
Syllable { // Why does these not work?
  delim = " -- ";
  midText = <<delim?" ";
  // emit the syllable
  // o = Output{line = ^midText + ^str;}
}

LyricsMelody_Phase2 {

  Lyrics | LyricLine | ending = Output{line = "\n";}

  melody = LilypondPlainMelody {
    title = "A Melody with lyrics";
    paperSize = "a6";
    paperOrientation = "landscape";
    contents = Melody {
    }
    lyricscontents = Output {line = ^lyricsText;}

//    Melody | lines = Empty {}
//    lyricscontents = Lyrics {
//      lineLengths = [16,16,16];
//    }
  }
}

/*** End of File ***/
```

The result: Running the specification test

And here is the result of the "specification" test:



Step 4: Generating marble models from marble - The marblemod$_{marble.marble}$ module.

So now two things are needed:

Now what is left is to let the `Lyrics` model generate a `LyricsMelody_Phase2` model (like shown in the `lyricsmelody_phase2_test` example).

This follows the exact same pattern as when making the `Melody` generating the audio, the score or the object graph: We need the marble model facilitating generating marble modules, and the extension to the lyrics that utilizes this module. This last one will be the tip-level "main program".

marblemod$_{marble.marble}$ Necessary features:

- Creating the file, the system class, including the include.

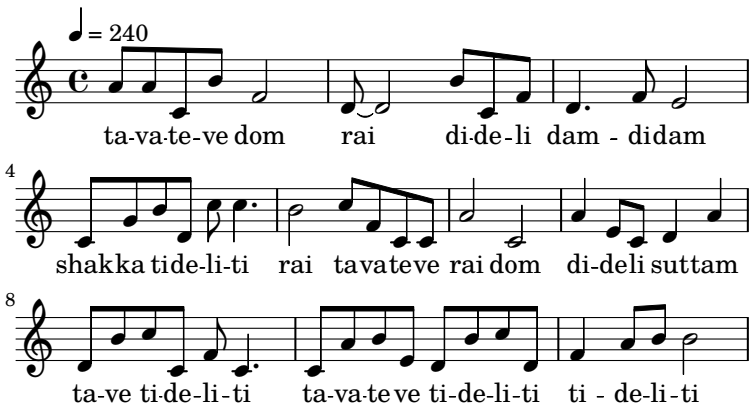
```
/*** #file "marblemod_marble.marble" ***/

Empty {}


VarAssignment {
  xlhs = "lhs";
  xrhs = "0";

  o1 = Output{line = "  " + ^xlhs + " = " + ^xrhs + ";\n";}
}

IncObjAssignment {
  xlhs = "lhs";
  xrhs = "Empty {}";

  o1 = Output{line = "  " + ^xlhs + " = " + ^xrhs + "\n";}
}



MarbleModel {
  modelName = "model";

  o1 = Output{line = "Include lyricsmelody_phase2.marble\n\n";}

  o1 += Output{line = "System {\n";}
  o1 += Output{line = "  Collection(collection,*|Output|line);\n\n";}

  incAssignments = Empty {}


  o2 += Output{line = "\n  m = LyricsMelody_Phase2 {}\n";}

  o2 += Output{line = "  \n}\n";}
```

```
}

/*** End of File ***/
```

Generating a phase2 model from Lyrics

```
/*** #file "lyricsmelody.marble" ***/
Include lyrics.marble
Include marblemod_marble.marble

Empty {}

System {
  Collection(collection,*|Output|line);

  mm = MarbleModel {
    modelName = "lyricsmelody_phase2";

    * | Lyrics | lineLengths = [12,12,12,12];

    incAssignments += VarAssignment {xlhs = "wordWeights"; xrhs = "[]";}
    incAssignments += Lyrics(123,456) {
      * | Word | o = VarAssignment {
        xlhs = "wordWeights";
        xrhs = "appendArray(wordWeights,[" + ^thisWord[1] + "])";
      }
    }

    incAssignments += VarAssignment {xlhs = "lyricsText"; xrhs = "\"\"";}
    incAssignments += Lyrics(123,456) {
      * | Syllable | o = VarAssignment {
        xlhs = "lyricsText"; xrhs = "lyricsText + \"" + ^midText + ^str + "\"";
      }
    }
  }
}
/*** End of File ***/
```

Here is the resulting melody:



Summary: Selling points

- The provided abstraction
- The deviances from hierarchical models.

Appendix: The dot/graphwiz model

```
/*** #file "marblemod_graphviz.marble" ***/

DotModel {
//  model = "model";
  title = "A Diagram";

  o += Output {line="/*** #file \""+ ^model + ".gv\" ***/\n";}
  o += Output {line="digraph \"unix\" {\n";}
  o += Output {line="  graph [        \n";}
  o += Output {line="    fontname = \"Helvetica-Oblique\",\n";}
  o += Output {line="    fontsize = 36,\n";}
  o += Output {line="    label = \"\n" + ^title + "\",\n";}
  o += Output {line="    size = \"100,100\" \n";}
  o += Output {line="  ];\n";}
  o += Output {line="  node [\n";}
  o += Output {line="    shape = polygon,\n";}
  o += Output {line="    sides = 4,\n";}
  o += Output {line="    distortion = \"0.0\",\n";}
  o += Output {line="    orientation = \"0.0\",\n";}
  o += Output {line="    skew = \"0.0\",\n";}
  o += Output {line="    color = white,\n";}
  o += Output {line="    style = filled,\n";}
  o += Output {line="    fontname = \"Helvetica-Outline\" \n";}
  o += Output {line="  ];\n";}
```

```
  o += Output {line="  /*** graphviz " + ^model + " nodes ***/\n";}
  o += Output {line="  /*** graphviz " + ^model + " arrows ***/\n";}
  o += Output {line="}\n";}
  o += Output {line="/*** End of File ***/\n";}
}

DotArrow {
//  model = "model";
  to = "uninitialized";
  from = "uninitialized";
  color = "black";

  o += Output {line="/*** graphviz " + ^model + " arrows ***/\n";}
  o += Output {line="\"" + ^from + "\" -> \"" + ^to + "\" [color=" + ^color + "]\n";}
  o += Output {line="/*** End of graphviz " + ^model + " arrows ***/\n";}
}

DotNode {
//  model = "model";
  name = "uninitialized";
  color = "white";

  o += Output {line="/*** graphviz " + ^model + " nodes ***/\n";}
  o += Output {line="\"" + ^node + "\" [color=" + ^color + "]\n";}
  o += Output {line="/*** End of graphviz " + ^model + " nodes ***/\n";}
}
/*** End of File ***/
```

Appendix: The Makefile

```
/*** #file "Makefile" ***/
all: marbledoc_soundtutorial.ok \
     melodyscore.pdf melody_dotmodel.png selfsimilar_melodyscore.pdf lyricstest.pdf\
     lyricsmelody_phase2_test.pdf lyricsmelody_phase2_test2.pdf

allsounds: all playmelody.wav playtuningfork.wav instrumenttest.wav instrumentmelody.wav\
           sameinstrumentmelody.wav

instrumenttest.wav: instrumenttest.marble instrument.marble melody.marble

instrumentmelody.wav: instrumentmelody.marble instrument.marble melody.marble

sameinstrumentmelody.wav: sameinstrumentmelody.marble instrument.marble melody.marble
```

```
marbledoc_soundtutorial.ok: ../marbledoc_soundtutorial.org
        clipprep $< | tinyclip -va
        touch $@

lyricsmelody_phase2_test2.marble: lyricsmelody.marble marblemod_marble.marble
        marbleapp_echo -f lyricsmelody.marble > $@

%.wav: %.marble
        marbleapp_sound -f $<

playmelody.wav: playmelody.marble
        marbleapp_sound -t 4 -f $<

%.ly: %.marble marbledoc_soundtutorial.ok
        marbleapp_echo -f $< >$@

melody_dotmodel.gv: melody_dotmodel.marble marbledoc_soundtutorial.ok
        marbleapp_echo -f $< | tinyclip -va

%.pdf:%.ly marbledoc_soundtutorial.ok
        lilypond -dbackend=eps -dno-gs-load-fonts -dinclude-eps-fonts $<

%.png:%.gv marbledoc_soundtutorial.ok
        dot -Tpng $< > $@
/*** End of File ***/
```